

Computer Vision Lab Python and OpenCV Cheat Sheet

Readable exam reference • Labs 01–06 • A4 print layout

Start with the shared setup below. In later snippets, `img` is a BGR uint8 image, `gray` is its grayscale version, and `h/w` are its height/width. Replace placeholder paths and choose parameters for the actual image. Code examples are templates, so variables such as `tx`, `ty`, and output dimensions must be set for your case.

Source note: Lab 02 point-transform topics are from the teacher announcement and the available Lab 02 tasks; its manual was unavailable. The Lab 01 manual was not in the accessible attachments, so Lab 01 follows its task sheet.

Shared setup and image coordinates

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

path = 'local_image.png'
img = cv2.imread(path, cv2.IMREAD_COLOR)
if img is None:
    raise FileNotFoundError(path)
h, w = img.shape[:2]
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(rgb); plt.axis('off'); plt.show()
```

Expression	Meaning
<code>img.shape[:2]</code>	(height, width); image indexing is [row, column] = [y, x].
Point (x, y)	OpenCV drawing and transform coordinates use x first, y second.
<code>cv2.resize(img, (W, H))</code>	Target size is (width, height), unlike NumPy shape.
<code>cv2.imwrite(path, img)</code>	Write a BGR image directly. Check the returned bool if saving matters.

Watch for: `cv2.imread` returns `None` for a bad path. OpenCV color images are BGR; convert to RGB before Matplotlib. Use `cmap="gray"` for one-channel arrays.

Lab 01 Python foundations and basic image operations

Python containers and a small class

A dictionary maps keys to values. A nested dictionary stores a record per key. A class groups data and methods. The task sheet uses these patterns before image processing.

```
records = {'S1': {'name': 'A', 'major': 'AI', 'grades': [84, 90]}}
valid = {k: v for k, v in records.items() if v['grades']}
best_id = max(valid, key=lambda k: np.mean(valid[k]['grades']))

class Inventory:
    def __init__(self): self.items = {}
    def add(self, name, qty):
        self.items[name] = self.items.get(name, 0) + qty
    def remove(self, name):
        if name not in self.items: raise KeyError(name)
        del self.items[name]
```

Use `items.get(key, default)` to avoid a missing-key error. Check for empty grade lists before calculating a mean.

Canvas, shapes and captions

A canvas is a uint8 array of zeros. OpenCV colors are BGR tuples. Thickness -1 fills a shape; positive thickness draws an outline.

```
canvas = np.zeros((h, w, 3), dtype=np.uint8)
cv2.circle(canvas, (w//2, h//2), 60, (0, 255, 0), 2)
cv2.rectangle(canvas, (20, 20), (150, 100), (255, 0, 0), 2)
cv2.putText(canvas, 'Target', (20, 18),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255,255,255), 2)

overlay = img.copy()
cv2.rectangle(overlay, (0, int(.8*h)), (w,h), (180,40,20), -1)
caption = cv2.addWeighted(img, .7, overlay, .3, 0)
```

addWeighted computes $\alpha \cdot \text{image1} + \beta \cdot \text{image2} + \gamma$. Both inputs must have the same size and type. The final argument 0 is a brightness offset.

Blur, ROI, threshold and masks

```
blur = cv2.GaussianBlur(img, (5,5), 0) # odd kernel
cx, cy, r = w//2, h//2, min(h,w)//6
roi = img[max(0,cy-r):min(h,cy+r), max(0,cx-r):min(w,cx+r)]
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
local = cv2.adaptiveThreshold(gray, 255,
                             cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)
mask = np.zeros((h,w), np.uint8)
cv2.circle(mask, (cx,cy), r, 255, -1)
foreground = cv2.bitwise_and(img, img, mask=mask)
background = cv2.bitwise_and(blur, blur, mask=cv2.bitwise_not(mask))
combined = cv2.bitwise_or(foreground, background)
```

threshold returns (chosen_threshold, output). For global threshold, a pixel above 127 becomes 255. adaptiveThreshold computes local thresholds; blockSize=11 is an odd neighborhood width and C=2 is subtracted from the local mean/weighted mean.

Quick image statistics

```
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
flat = rgb.reshape(-1, 3)
print('mean R,G,B:', flat.mean(axis=0))
print('min/max:', flat.min(axis=0), flat.max(axis=0))
print('std:', flat.std(axis=0))
```

Watch for: NumPy slices exclude their end index. Masks are one-channel uint8 with 0 for excluded and 255 for included pixels. Heavy blur can erase fine detail.

MCQ checkpoint

- **BGR versus RGB:** cv2.imread returns BGR; Matplotlib displays RGB. The conversion changes channel order, not pixel positions.
- **ROI order:** img[y0:y1, x0:x1] selects rows before columns; both end indices are excluded.
- **Thresholds:** Global uses one cutoff; adaptive calculates a local cutoff. Gaussian kernels and adaptive windows use positive odd sizes.

Lab 02 Photometric and point transformations

A point transformation changes a pixel value without moving its location. The teacher announced log, gamma, piecewise, and histogram topics. The available Lab 02 task sheet also uses equalization, false color, weighted fusion, thresholding and video.

Method	What it does	When to try it
Log	Expands differences among dark intensities.	Underexposed areas.
Gamma	Power curve on normalized pixels; gamma below 1 brightens.	Tune midtones.
Piecewise	Different slopes in selected intensity intervals.	Emphasize a chosen range.
Histogram equalization	Uses cumulative intensity distribution to spread contrast.	Low-contrast grayscale images.

Log, gamma and piecewise mappings

```
f = gray.astype(np.float32) / 255.0
log_img = np.uint8(np.clip(
    255*np.log1p(gray.astype(np.float32))/np.log(256), 0, 255))
gamma = 0.65
power_img = np.uint8(np.clip(255*(f**gamma), 0, 255))
# Input knots and output knots must be sorted left to right.
piecewise = np.interp(gray, [0, 70, 160, 255],
    [0, 30, 220, 255]).astype(np.uint8)
```

For normalized intensities between 0 and 1: gamma below 1 brightens; gamma above 1 darkens. In the piecewise map, the segment from input 70 to 160 has a steep slope and increases contrast there. Clip before uint8 conversion to avoid wraparound.

Histogram, equalization, false color and threshold

```
hist = cv2.calcHist([gray], [0], None, [256], [0,256])
plt.plot(hist); plt.xlabel('intensity'); plt.ylabel('pixel count')
equal = cv2.equalizeHist(gray)
heat_bgr = cv2.applyColorMap(equal, cv2.COLORMAP_JET)
_, high_mask = cv2.threshold(equal, 190, 255, cv2.THRESH_BINARY)
```

calcHist: [image], [channel index], optional mask, [number of bins], and intensity range. equalizeHist needs a single-channel uint8 image. False color maps the same grayscale values to colors; it does not create new image information.

Color balance, aligned fusion and video

```
# Simple channel-gain correction on a displayed BGR heatmap.
gains = np.array([1.0, .95, 1.05], np.float32)
balanced = np.uint8(np.clip(heat_bgr.astype(np.float32)*gains,0,255))
# CT and MRI must already be registered and have identical shape/type.
ct2, mri2 = cv2.equalizeHist(ct_gray), cv2.equalizeHist(mri_gray)
fused = cv2.addWeighted(ct2, .65, mri2, .35, 0)

cap = cv2.VideoCapture('local_clip.mp4')
if not cap.isOpened(): raise FileNotFoundError('local_clip.mp4')
while True:
    ok, frame = cap.read()
    if not ok: break
    enhanced = cv2.equalizeHist(cv2.cvtColor(frame,cv2.COLOR_BGR2GRAY))
    # display or save enhanced frame here
    cap.release()
```

addWeighted weights alpha and beta combine the two aligned inputs; gamma is the scalar offset. In a video loop, stop at a failed read and release the capture. cv2.imshow additionally needs cv2.waitKey, and GUI display may be unavailable in notebook environments.

Watch for: Equalization can amplify noise. A resize does not register mismatched anatomy. Do not interpret false-color heatmaps as measured color data.

MCQ checkpoint

- **Point versus geometric:** Photometric operations change intensity at a location; geometric operations change where a pixel appears.
- **Gamma:** For intensities normalized to 0–1, gamma below 1 brightens and gamma above 1 darkens. Gamma=1 is unchanged.
- **Histogram:** A histogram counts pixels by intensity. Equalization redistributes contrast through its cumulative distribution and can amplify noise.
- **Fusion:** Weighted fusion requires images of the same shape and aligned content; a heatmap is a visualization, not extra measured data.

Lab 03 Geometric transformations

Geometric transformations move pixels to new coordinates. A 2×2 linear matrix can scale, rotate or shear around the origin. A 3×3 homogeneous matrix also represents translation. Projective warps need a full 3×3 homography.

Type	Preserves	OpenCV operation
Rigid	Lengths and angles; rotation + translation.	warpAffine
Similarity	Angles and shape; uniform scale allowed.	warpAffine
Affine	Straight and parallel lines.	warpAffine
Projective	Straight lines; parallelism may change.	warpPerspective

Compose scale, rotation and translation

```

sx, sy, degrees, tx, ty = 1.2, 1.2, 20, 30, -15
a = np.deg2rad(degrees)
S = np.array([[sx,0,0],[0,sy,0],[0,0,1]], np.float32)
R = np.array([[np.cos(a),-np.sin(a),0],
              [np.sin(a), np.cos(a),0],[0,0,1]], np.float32)
T = np.array([[1,0,tx],[0,1,ty],[0,0,1]], np.float32)
M = T @ R @ S          # scale, then rotate, then move
result = cv2.warpAffine(img, M[:2], (w,h))

```

The rightmost matrix acts first for column-vector points. For a rigid transform use $s_x=s_y=1$. For similarity use $s_x=s_y$ with another value. For general affine, s_x and s_y can differ and shear may be added.

Rotate without clipping

```

angle = 30
M = cv2.getRotationMatrix2D((w/2,h/2), angle, 1.0)
c, s = abs(M[0,0]), abs(M[0,1])
new_w, new_h = int(h*s+w*c), int(h*c+w*s)
M[0,2] += new_w/2 - w/2
M[1,2] += new_h/2 - h/2
rotated = cv2.warpAffine(img, M, (new_w,new_h))

```

`getRotationMatrix2D` takes center, angle in degrees, scale. `warpAffine` takes a 2×3 matrix and output size (width,height). Enlarging the canvas and shifting its center preserves corners.

Estimate affine or perspective mapping from points

```

src3 = np.float32([[0,0],[w-1,0],[0,h-1]])
dst3 = np.float32([[20,10],[w-20,5],[10,h-10]])
A = cv2.getAffineTransform(src3, dst3)
affine = cv2.warpAffine(img, A, (w,h))

src4 = np.float32([[10,20],[w-20,5],[w-1,h-10],[0,h-1]])
W,H = 400,300
dst4 = np.float32([[0,0],[W-1,0],[W-1,H-1],[0,H-1]])
P = cv2.getPerspectiveTransform(src4,dst4)
rectified = cv2.warpPerspective(img,P,(W,H))

```

Use three non-collinear corresponding pairs for affine and four for perspective. Keep corner order consistent, for example top-left, top-right, bottom-right, bottom-left. A homography estimated from many noisy matches uses `cv2.findHomography` with RANSAC instead.

Watch for: A 2×2 matrix cannot translate. The output canvas may clip transformed pixels. Nearest-neighbor interpolation is useful for binary masks; linear is the usual image default.

MCQ checkpoint

- **Preservation:** Rigid preserves distance and angles; similarity preserves angles but can change size; affine preserves parallelism; projective need not preserve parallelism.
- **Matrix size:** A 2×2 linear map cannot translate. A 2×3 affine map can. Perspective uses 3×3 .
- **Point count:** Affine estimation needs three non-collinear pairs; perspective needs four. Corresponding corner order must agree.

Lab 04 Feature extraction, filtering and edges

A feature is a compact measurement of an image region: color, texture, edge direction or intensity pattern. A feature vector can support matching or classification.

Feature	Meaning / use
Color histogram	Counts pixels by color or intensity bin; ignores location.
HOG / HED	Histograms of gradient/edge directions; captures shape.
HIG	Histogram of gradient strength; captures intensity-change sizes.
LBP	Compares neighboring pixels with a center pixel; describes texture.
Texture energy / contrast	Summaries of variation or filter response across a region.

Histograms and a simple gradient-orientation descriptor

```
hist_gray = cv2.calcHist([gray],[0],None,[256],[0,256])
hist_blue = cv2.calcHist([img],[0],None,[256],[0,256])
gx = cv2.Sobel(gray,cv2.CV_64F,1,0,ksize=3)
gy = cv2.Sobel(gray,cv2.CV_64F,0,1,ksize=3)
mag = cv2.magnitude(gx,gy)
angle = cv2.phase(gx,gy,angleInDegrees=True) % 180
orientation_hist, _ = np.histogram(angle,bins=9,
                                   range=(0,180),weights=mag)
```

Sobel $dx=1, dy=0$ measures changes across x; $dx=0, dy=1$ measures changes across y. CV_64F preserves negative gradients. Magnitude is the strength; angle is the direction. A full HOG descriptor also divides the image into spatial cells and normalizes blocks.

Filtering and convolution

```
box_kernel = np.ones((3,3),np.float32)/9
box = cv2.filter2D(gray,-1,box_kernel)
gaussian = cv2.GaussianBlur(gray,(5,5),0)
median = cv2.medianBlur(gray,5)
laplacian = cv2.Laplacian(gaussian,cv2.CV_64F)
```

filter2D applies a custom kernel. ddepth=-1 retains the input depth; choose CV_64F when signed responses matter. Gaussian reduces Gaussian-like noise; median is useful for impulse noise. Valid convolution has no padding and shrinks the image; same padding keeps the dimensions.

Canny edge detection

```
smooth = cv2.GaussianBlur(gray,(5,5),0)
edges = cv2.Canny(smooth,80,160)
```

Canny applies smoothing, gradients, nonmaximum suppression, then two-threshold hysteresis. Pixels above the high threshold are strong edges; weak edges between thresholds survive if linked to strong edges. Raising thresholds removes weak edges and noise, but may miss real boundaries.

Local binary pattern template

```
center = gray[1:-1,1:-1]
lbp = np.zeros_like(center)
for bit,(dy,dx) in enumerate([( -1,-1),(-1,0),(-1,1),(0,1),
                             (1,1),(1,0),(1,-1),(0,-1)]):
    neighbor = gray[1+dy:h-1+dy,1+dx:w-1+dx]
    lbp |= ((neighbor>=center).astype(np.uint8) << bit)
```

Each neighbor comparison contributes one bit to an 8-bit texture code. The output omits the one-pixel border because a full neighborhood is unavailable there.

Watch for: Do not cast raw signed Sobel values to uint8. Use abs or magnitude first. HOG and LBP are feature descriptions, not segmentation masks.

MCQ checkpoint

- **Feature versus mask:** A feature summarizes image properties; a segmentation mask assigns pixels to a region.
- **Sobel:** X and Y derivatives are signed. CV_64F preserves negative gradients; magnitude combines directions.
- **Canny:** Smoothing, gradient, nonmaximum suppression and hysteresis produce thin linked edges. A high threshold finds strong edges; a low threshold admits connected weak edges.
- **Padding:** Valid convolution reduces output dimensions; same padding maintains them.

Lab 05 Image segmentation

Segmentation assigns pixels to meaningful regions. Choose a method based on the image: intensity, illumination, color, connectivity, touching objects or multiple color groups.

Method	Assumption	Important parameter
Global threshold	One intensity cut separates foreground.	threshold T
Adaptive	Illumination changes spatially.	odd blockSize, C
Otsu	Histogram has separable intensity classes.	threshold chosen automatically
HSV range	Target has distinct hue/saturation.	lower and upper HSV bounds
Region growing	Target is connected near a trusted seed.	seed and tolerance
Watershed	Touching objects have separate cores.	foreground distance cutoff
K-means	Similar colors can form groups.	number of clusters K

Global, adaptive, Otsu and HSV masks

```
_, fixed = cv2.threshold(gray,120,255,cv2.THRESH_BINARY)
local = cv2.adaptiveThreshold(gray,255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,cv2.THRESH_BINARY,21,5)
otsu_T, otsu = cv2.threshold(gray,0,255,
    cv2.THRESH_BINARY+cv2.THRESH_OTSU)
hsv = cv2.cvtColor(img,cv2.COLOR_BGR2HSV)
low = np.array([20,80,60]); high = np.array([40,255,255])
color_mask = cv2.inRange(hsv,low,high)
selected = cv2.bitwise_and(img,img,mask=color_mask)
```

For adaptiveThreshold, blockSize=21 is the odd local window and C=5 is subtracted from the local threshold. Compare mean and Gaussian methods. Otsu ignores the supplied 0 threshold and returns its selected value. In OpenCV uint8 HSV, hue spans 0–179; saturation/value span 0–255.

Region growing from a seed

```
from collections import deque
def grow(a, seed_yx, tolerance):
    h,w = a.shape; sy,sx = seed_yx
    target = int(a[sy,sx]); mask = np.zeros((h,w),np.uint8)
    queue = deque([(sy,sx)]); mask[sy,sx] = 255
    while queue:
        y,x = queue.popleft()
        for ny,nx in ((y-1,x),(y+1,x),(y,x-1),(y,x+1)):
            if (0<=ny<h and 0<=nx<w and mask[ny,nx]==0
                and abs(int(a[ny,nx])-target)<=tolerance):
                mask[ny,nx]=255; queue.append((ny,nx))
    return mask
region = grow(gray,(h//2,w//2),20)
```

The seed is (row,column). A neighbor joins if its intensity is close enough to the seed intensity. A larger tolerance grows farther and may leak into nearby tissue/background. Changing the seed may select a different connected region.

Marker-based watershed for touching objects

```
# Start from a binary mask with objects white (255).
binary = otsu # invert first if objects are black
kernel = np.ones((3,3),np.uint8)
clean = cv2.morphologyEx(binary,cv2.MORPH_OPEN,kernel,iterations=2)
sure_bg = cv2.dilate(clean,kernel,iterations=3)
dist = cv2.distanceTransform(clean,cv2.DIST_L2,5)
_, sure_fg = cv2.threshold(dist,4*dist.max(),255,0)
sure_fg = sure_fg.astype(np.uint8)
unknown = cv2.subtract(sure_bg,sure_fg)
_, markers = cv2.connectedComponents(sure_fg)
markers += 1; markers[unknown==255] = 0
labels = cv2.watershed(img.copy(),markers.astype(np.int32))
outline = img.copy(); outline[labels==-1] = (0,0,255)
```

Opening removes small noise. Dilation estimates sure background. The distance map peaks near object centers. Thresholding it creates separate sure-foreground cores. Unknown pixels have marker 0; watershed grows labels until regions meet. Output label -1 is a boundary.

K-means color grouping

```
Z = img.reshape(-1,3).astype(np.float32)
criteria=(cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX_ITER,20,.2)
_, labels, centers = cv2.kmeans(Z,4,None,criteria,10,
                               cv2.KMEANS_PP_CENTERS)
segmented=np.uint8(centers[labels.flatten()]).reshape(img.shape)
```

K=4 means four representative colors; attempts=10 restarts the clustering. Reconstructed colors are cluster centers. More K preserves more detail but creates more regions. A color cluster is not necessarily one connected object.

Watch for: Binary polarity matters for watershed: objects must be white. Check masks visually before later steps. If a histogram is not roughly bimodal, Otsu may choose an unhelpful threshold.

MCQ checkpoint

- **Threshold selection:** Global needs a chosen T; Otsu chooses T from the intensity distribution; adaptive varies T over local neighborhoods.
- **HSV:** Hue is 0–179 in OpenCV uint8 HSV. inRange includes values between the lower and upper bounds.
- **Watershed:** Sure foreground and sure background define markers; unknown=0 and final boundary=-1. A distance cutoff changes how many object cores survive.
- **K-means:** K is the number of color centers, not necessarily the number of connected objects.

Lab 06 Wavelets, Hough transforms and SIFT

Wavelet idea and an offline Haar example

A wavelet transform splits a signal into coarse approximation and local detail at a chosen scale. Large detail coefficients can mark rapid changes such as spikes. A discrete wavelet transform (DWT) handles sampled data; continuous wavelets explore many scales. Inverse transform reconstructs the signal.

```
signal = np.asarray(signal, dtype=float) # 1D local samples
x = signal[(len(signal)//2)*2] # Haar pairs need even length
approx = (x[0::2]+x[1::2])/np.sqrt(2)
detail = (x[0::2]-x[1::2])/np.sqrt(2)
med = np.median(detail)
mad = np.median(np.abs(detail-med)) + 1e-9
anomaly_pairs = np.flatnonzero(np.abs(detail-med)>6*1.4826*mad)
```

Approximation averages each pair; detail is their difference. The median/MAD threshold is robust to a few spikes. Tune the multiplier: lower detects more anomalies and more false alarms. For full multilevel DWT or reconstruction, use a locally installed wavelet package if available; this Haar template needs only NumPy.

Hough line detection

Hough voting finds shapes from edge pixels. Probabilistic Hough lines return endpoint pairs, which are easy to draw and filter by length or slope.

```
edges = cv2.Canny(gray,60,160)
lines = cv2.HoughLinesP(edges,1,np.pi/180,60,
                        minLineLength=35,maxLineGap=10)
line_view = img.copy()
if lines is not None:
    for x1,y1,x2,y2 in lines[:,0]:
        cv2.line(line_view,(x1,y1),(x2,y2),(0,255,0),2)
```

$\rho=1$ is distance resolution in pixels; $\theta=\pi/180$ is angle resolution; $\text{threshold}=60$ is required votes. `minLineLength` rejects short segments; `maxLineGap` joins nearby collinear segments.

Hough circle detection

```
soft = cv2.medianBlur(gray,5)
circles = cv2.HoughCircles(soft,cv2.HOUGH_GRADIENT,
    dp=1.2,minDist=25,param1=120,param2=25,
    minRadius=8,maxRadius=70)
if circles is not None:
    for x,y,r in np.round(circles[0]).astype(int):
        cv2.circle(img,(x,y),r,(0,255,0),2)
```

`dp` is accumulator resolution ratio; `minDist` separates centers; `param1` is the internal Canny high threshold; `param2` is the circle-center vote threshold. A lower `param2` finds more circles, including false positives.

SIFT feature matching and a homography

SIFT detects local keypoints across scales, assigns orientation and creates 128-value descriptors. Match descriptors using L2 distance; use the ratio test to remove ambiguous matches and RANSAC to reject geometrically inconsistent ones.

```
sift = cv2.SIFT_create()
kp1,d1 = sift.detectAndCompute(ref_gray,None)
kp2,d2 = sift.detectAndCompute(scene_gray,None)
if d1 is not None and d2 is not None and len(d2)>=2:
    pairs=cv2.BFMatcher(cv2.NORM_L2).knnMatch(d1,d2,k=2)
    good=[m for pair in pairs if len(pair)==2
          for m,n in [pair] if m.distance < .75*n.distance]
    if len(good)>=4:
        p1=np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1,1,2)
        p2=np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1,1,2)
        H,inliers=cv2.findHomography(p1,p2,cv2.RANSAC,5.0)
    if H is not None:
        rh,rw=ref_gray.shape
        corners=np.float32([[0,0],[rw-1,0],[rw-1,rh-1],
                           [0,rh-1]]).reshape(-1,1,2)
        scene_corners=cv2.perspectiveTransform(corners,H)
```

The ratio .75 requires the nearest match to be clearly better than the second nearest. `findHomography` requires at least four matched pairs. The RANSAC threshold 5.0 is maximum reprojection error in pixels for an inlier. Use `scene_corners` to draw a detected quadrilateral or warp for stitching.

Watch for: Hough calls can return None. SIFT may be unavailable in older OpenCV builds. Always check descriptor and match counts before estimating a homography.

MCQ checkpoint

- **Hough:** Line detection uses edge votes in line-parameter space; circle detection votes for center and radius. Higher vote thresholds usually return fewer detections.
- **SIFT:** Scale-space keypoints and orientation support scale/rotation changes; the descriptor has 128 values. Ratio filtering and RANSAC reject different kinds of bad matches.
- **Wavelets:** Approximation holds coarse behavior; detail holds local changes. Large detail or residual values can indicate anomalies, depending on threshold.

Fast lookup Choose the method and check the output

Image problem	First method to try	Inspect / tune
Bad path or odd colors	imread check; BGR to RGB for plotting	None, color conversion
Dark compressed detail	log or gamma below 1	clipping, noise
Uneven lighting	adaptive threshold	blockSize, C
Two grayscale classes	Otsu threshold	histogram shape
Distinct color	HSV inRange	hue bounds, saturation
Touching objects	distance transform + watershed	foreground cutoff and markers
Many color regions	K-means	K, connectedness
Straight lines or circles	HoughLinesP / HoughCircles	vote thresholds and geometry
Rotated or scaled object match	SIFT + ratio + RANSAC	good matches, inliers
Short sensor spikes	wavelet detail + robust threshold	threshold vs false alarms

Ten checks before submitting code

- **1. Loading** Check img is not None before shape or cvtColor.
- **2. Color** Keep BGR for OpenCV; convert to RGB for Matplotlib.
- **3. Coordinates** NumPy uses [y,x]; OpenCV points use (x,y).
- **4. Size** resize and warp output sizes are (width,height).
- **5. Type** Use uint8 for ordinary images/masks, float32 for kmeans, int32 for watershed markers.
- **6. Threshold** Capture both return values from cv2.threshold; Otsu returns its chosen threshold.
- **7. Masks** Single-channel, 0/255; invert with bitwise_not when needed.
- **8. Geometry** Corresponding points in matching order; enlarge canvas to prevent clipping.
- **9. Optional outputs** Guard None from Hough and SIFT descriptors; require at least four homography matches.
- **10. Output** Plot comparisons, save the result, and state parameter values used.

Exam scope reminder: the teacher announcement allows printed Python/OpenCV cheat sheets. This document contains Python/OpenCV definitions and code patterns; the separate practice bank and notebook are study aids.